

CLoMA-UWR: LLM-Centered Closed-Loop Multiagent Framework for Multi-UAV Wildfire Rescue

Author One and Author Two

Abstract—Reliable multi-UAV wildfire rescue requires closed-loop decision making that converts uncertain aerial observations into executable fleet plans and preserves mission continuity under runtime perturbations. Existing UAV approaches commonly optimize perception, navigation, or task allocation as separate capabilities, leaving a gap between fire-situation understanding, resource-constrained mission planning, and runtime replanning. To close this gap, we develop CLoMA-UWR, an LLM-centered closed-loop multiagent framework for multi-UAV wildfire rescue. CLoMA-UWR transforms aerial observations into initial mission plans through Vision Analysis and Mission Planning, audits candidate plans through Validation, uses Memory to inform subsequent decisions, and invokes Runtime Replanning after runtime perturbations. A real-UAV-informed AirSim suite evaluates it across four difficulty levels with resource limitations, scheduling constraints, and runtime perturbations. Across four downstream foundation-model configurations, CLoMA-UWR outperforms an Optimization-Based Planner in task completion and execution quality under hard and expert conditions. Controlled ablations identify the condition-dependent roles of Validation, Memory, and Runtime Replanning. On simple and normal episodes, a fine-tuned Qwen3-8B planner retains 98.24% of the Gemini 3.1 Pro teacher planner’s average score while reducing mean planning time from 37.50 to 14.01 s and mean planning-token usage from 13,353 to 3,593. These results demonstrate the effectiveness of CLoMA-UWR for adaptive multi-UAV wildfire rescue. The implementation is available at <https://github.com/tongsuhdyem/Pipeline>.

Index Terms—Closed-loop decision making, large language model agents, multi-UAV systems, real-to-simulation evaluation, wildfire rescue.

I. INTRODUCTION

WILDFIRES threaten ecosystems, infrastructure, and human safety [1]. In wildfire emergency response, UAVs can rapidly observe hazardous and inaccessible areas through flexible aerial sensing [2]. Turning this capability into an effective field operation, however, requires observations to be converted into task zones, assigned across a resource-limited fleet, and revised as the fire or UAV state changes. Multi-UAV wildfire rescue is therefore a closed-loop decision problem rather than an isolated perception or path-planning task.

An initial mission must be feasible for the current fleet: task priorities must be matched to vehicle locations, endurance, payload, communication, and safety limits [3]. Feasibility is also temporary because fire evolution, resource consumption, communication changes, and UAV faults can invalidate existing

assignments. Initial resource-constrained mission planning and runtime replanning must consequently be treated as coupled parts of the same decision problem.

Existing work has advanced the individual capabilities needed for this process [4]. Thermal–RGB fusion, aerial semantic segmentation, and learning-based 3D reconstruction improve fire detection and scene modeling [5]–[7], while autonomous exploration and mapping and next-best-view planning update environmental knowledge and reduce observation uncertainty [8], [9]. On the action side, deep reinforcement learning and coverage path planning support navigation and large-area search [10], [11], whereas multi-agent reinforcement learning and human–robot collaboration support task allocation and search-and-rescue operations [12], [13]. However, these capabilities are commonly optimized and evaluated separately, often from a fixed initial state. They therefore do not establish how visual and spatial evidence can be converted into fire points and task zones for resource-constrained mission planning, or how an initially feasible allocation should be revised when the fire, fleet, or resource state changes.

A high-level decision layer must convert structured observations into executable fleet plans. Large language models (LLMs) provide a flexible interface for this role: SayCan grounds instructions in executable skills [14], Code as Policies generates programmatic control policies [15], and related design guidance exposes tools and environmental constraints to ChatGPT-generated robot workflows [16]. Embodied and multimodal models further connect reasoning to physical state: RT-1 learns vision-language-action policies [17], PaLM-E incorporates visual inputs and robot states [18], and Socratic Models compose specialized vision and language models [19]. These approaches establish useful mechanisms for task decomposition, action sequencing, and physical grounding, but are generally evaluated on individual robots and predefined manipulation or navigation tasks rather than resource-constrained mission planning for a multi-UAV fleet.

LLM-based agents have also used environmental feedback and accumulated experience for sequential decision making. Inner Monologue revises embodied plans from language-based feedback [20], LM-Nav uses semantic waypoints for open-world navigation [21], and Voyager combines an LLM agent with an automatic curriculum and a growing skill library [22]. These systems establish that feedback and accumulated experience can improve sequential behavior, but primarily target single-agent embodied environments rather than dynamic, interdependent tasks across a resource-constrained fleet. Existing

research therefore provides the main ingredients for perception, planning, coordination, and feedback, but leaves two operational requirements unresolved. First, linguistic coherence does not ensure that a mission plan jointly respects endurance, payload, communication, priority, and safety constraints. Second, an initially feasible mission does not ensure mission continuity after the fire, fleet, resource, or schedule state changes. A common high-level decision loop must connect fire-situation understanding and resource-constrained mission planning with Validation, Memory, and Runtime Replanning to address both requirements as operational pressure increases.

To address these requirements, we develop CLoMA-UWR, an LLM-centered closed-loop multiagent framework that transforms aerial observations through Vision Analysis and Mission Planning into an initial mission plan. Validation rejects plans that are tactically implausible or structurally invalid, converting validation failures into targeted feedback for Mission Planning. Memory retrieves relevant prior experience and guidance about recurring planning and resource-management patterns. Runtime Replanning repairs the affected part of the active mission after changes in the fire, fleet, resources, or schedule. Together, these mechanisms turn one-shot LLM planning into a decision process that connects fire-situation understanding, resource-constrained mission planning, Validation, execution feedback, and Runtime Replanning within one continuous loop.

We evaluate CLoMA-UWR in a real-UAV-informed AirSim suite through end-to-end comparisons, controlled ablations, and compact-planner fine-tuning.

The main contributions of this work are summarized as follows:

- 1) We formulate resource-constrained multi-UAV wildfire rescue as a high-level closed-loop decision problem and develop CLoMA-UWR, an LLM-centered closed-loop multiagent framework that connects fire-situation understanding and resource-constrained mission planning with Validation, Memory, and Runtime Replanning.
- 2) We construct a real-UAV-informed AirSim evaluation suite that exposes the decision loop to controlled increases in operational pressure through resource limitations, scheduling constraints, and runtime perturbations across four difficulty levels, and measures both task completion and execution quality with six mission-level metrics.
- 3) We evaluate end-to-end behavior against an Optimization-Based Planner and across multiple foundation-model configurations, characterize through controlled ablations how Validation, Memory, and Runtime Replanning contribute as operational pressure increases, and assess whether validated demonstrations can specialize a compact planner with lower planning overhead.

II. CLoMA-UWR FRAMEWORK

This section formulates multi-UAV wildfire rescue as a sequence from fire-situation understanding to resource-constrained mission planning and runtime replanning, and presents CLoMA-UWR as the framework that implements this sequence. The task model represents aerial observations, fire points, task zones, UAV operational states, mission constraints,

runtime perturbations, and the corresponding initial and revised mission plans. It exposes three decision risks: a generated plan may violate tactical or structural constraints, one-shot reasoning may fail to use relevant prior experience, and an initially feasible plan may become obsolete during execution. As shown in Figure 1, Vision Analysis and Mission Planning construct candidate plans, Validation checks them before execution, Memory supplies prior experience, and Runtime Replanning revises the active mission after operational state changes. The following subsections detail the task model and system components.

A. Task Modeling

For each mission, the rescue world is represented by the structured state S :

$$S = \{O, F, R, U, C, E\}.$$

Here, O denotes aerial observations; F denotes fire points; R denotes task zones; U denotes UAV operational states; C denotes mission constraints; and E denotes runtime perturbations. This representation separates the underlying rescue world from the information available to the decision system. At mission start, the system receives the initial input $X_0 = \{O, U, C\}$, whereas F and R must be inferred from the observation. When a runtime perturbation occurs, it receives the updated operational state U_t and the observed perturbation E_t . The required outputs are an initial mission plan P and, when necessary, a revised plan P' .

The resulting task model comprises the following decision chain:

$$\begin{aligned} O &\rightarrow F \rightarrow R, & (R, U, C) &\rightarrow P, \\ & & (P, U_t, E_t) &\rightarrow P'. \end{aligned}$$

The first two relations transform raw aerial observations into an actionable regional representation of the fire situation. The third relation generates an executable fleet plan under operational constraints. The final relation adapts that plan to changes encountered during execution. This formulation captures the continuous rescue workflow while retaining the stage-specific information needed to design and evaluate a closed-loop system.

B. Proposed Framework

Figure 1 summarizes the flow of mission state, candidate plans, corrective feedback, and prior experience among the CLoMA-UWR components, whose individual roles are described below.

a) Vision Analysis: Vision Analysis anchors the decision loop by reducing the ambiguity between raw aerial observations and the planning state. It converts raw aerial imagery into a structured description of the fire situation. This module uses a multimodal LLM to perform two tasks in a single inference pass. First, it detects all visible fire points in the image and outputs their normalized coordinates. Second, it clusters these points by spatial density, partitioning the continuous fire field into several discrete task zones.

This module uses a multimodal LLM rather than a vision-only model because zone division depends on more than fire

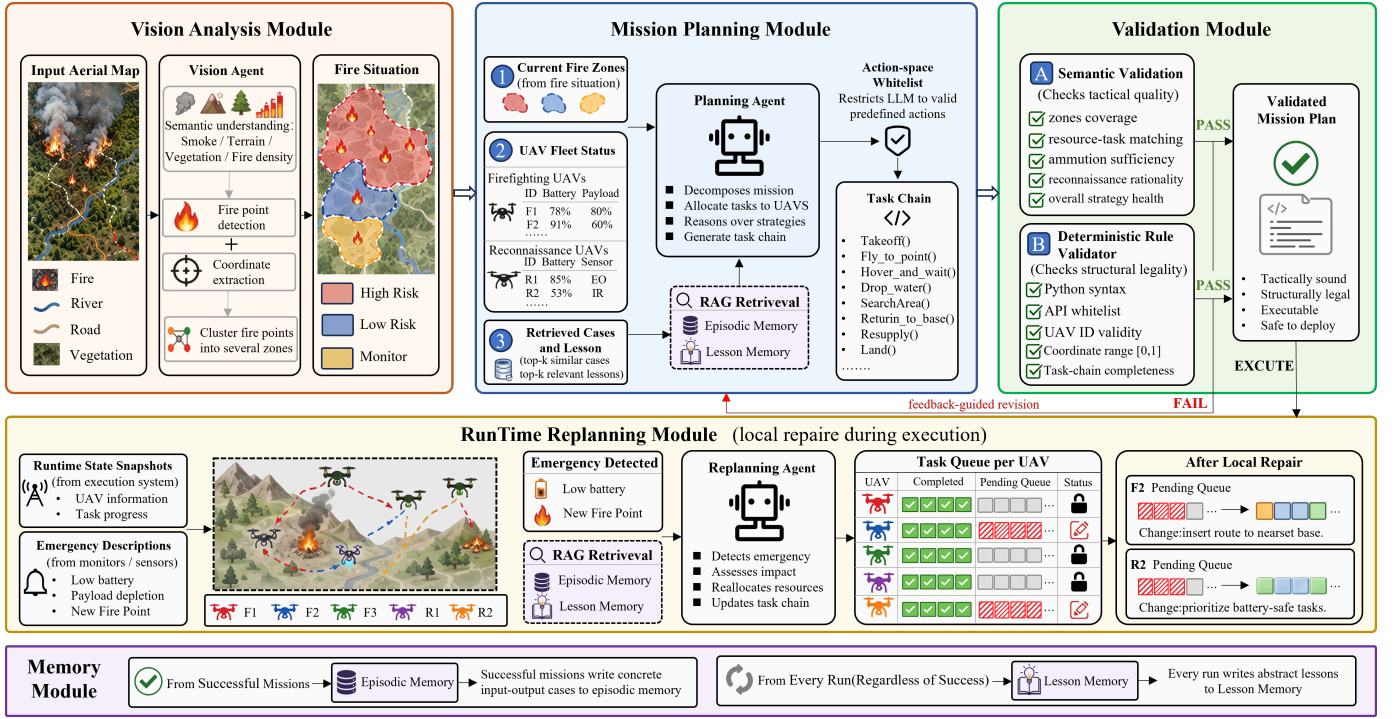


Fig. 1. Overview of CLOMA-UWR.

point locations. The division also depends on visual context, such as smoke, vegetation, and terrain boundaries. These cues require joint reasoning over appearance and spatial semantics. The resulting fire points and task zones provide the shared initial-state representation used by Mission Planning.

b) Mission Planning: Mission Planning addresses the fleet-level coupling among task priorities, vehicle states, and resource constraints. It is the core decision module and converts the structured fire situation produced by Vision Analysis into a candidate multi-UAV collaborative plan. A planning LLM generates a complete task chain for each UAV in the fleet, covering task allocation, temporal sequencing, and endurance management. To keep outputs controllable, the LLM is restricted to a predefined set of allowable actions.

Mission Planning combines the fire situation, retrieved experience, and operational constraints. It receives the structured fire situation from Vision Analysis, retrieves relevant prior experience from Memory, and produces a coordinated plan in which each UAV's task chain respects the fleet's capabilities and the mission's operational constraints. Formally, the planning process is expressed as

$$P = \mathcal{G}(R, U, C, M(q)), \quad q = q(R, U, C), \quad (1)$$

where $\mathcal{G}$ denotes the planning LLM, $q(R, U, C)$ denotes the query construction from the current task zones, fleet states, and constraints, and $M(q)$ denotes the prior experience retrieved from Memory for that query. The resulting P is a candidate plan rather than an automatically trusted output; it is passed to Validation before mission execution.

c) Validation: Validation addresses the risk that a linguistically coherent candidate plan may still be tactically implausible

or structurally invalid. It evaluates tactical rationality and structural legality, including zone coverage completeness, resource-task matching, payload adequacy, UAV existence, coordinate validity, and task-chain completeness. When Validation rejects a plan, its feedback is routed back to Mission Planning as targeted guidance for revision. Only an accepted plan proceeds to execution, preserving the separation between plan generation and plan acceptance.

d) Runtime Replanning: Runtime Replanning addresses the risk that an accepted plan becomes invalid after the operational state changes. This module handles runtime perturbations during mission execution. It takes a runtime state snapshot as input, including current positions, battery levels, payloads, and task progress for each UAV. Its output is not a new global plan, but a local revision of the current plan. Unlike the main pipeline, Runtime Replanning reasons about a dynamic execution state and changes only the affected part of the plan.

When a runtime perturbation occurs, the module retrieves similar prior cases from Memory as repair references. These cases describe how comparable failures were resolved in past missions, providing the replanning LLM with tactical precedents for the current situation. The resulting local revision closes the execution-feedback loop without restarting the complete pre-execution pipeline.

e) Memory: Memory addresses the risk that relevant prior experience is unavailable when coupled resource, scheduling, or event conditions make the current decision less routine. It allows the system to reuse past experience. Each memory entry records a task situation, the generated or revised plan, validation feedback, runtime perturbations, the execution outcome, and reusable guidance extracted from the completed mission.

TABLE I

COVERAGE OF EVALUATION CAPABILITIES IN EXISTING DATASETS, BENCHMARKS, AND THE EVALUATION SUITE USED IN THIS WORK.

● INDICATES EXPLICIT COVERAGE, ○ INDICATES PARTIAL COVERAGE, AND — INDICATES NO EXPLICIT COVERAGE.

Benchmark / dataset	Fire perc.	Disaster sem.	UAV constr.	Runtime repl.
DOTA [23]	●	—	—	—
VisDrone [24]	●	—	—	—
UAVs-FFDB [25]	●	●	—	—
FLAME [26]	●	●	—	—
MS-FSDB [27]	○	●	—	—
WildFireVQA [28]	●	●	○	—
RMASBench [29]	—	●	○	○
CrisisBench [30]	—	●	—	—
Our Evaluation Suite	●	●	●	●

Memory is represented as

$$M = \{(z_i, P_i, o_i)\}_{i=1}^N, \quad (2)$$

where $m_i = (z_i, P_i, o_i)$ denotes a memory entry, and z_i , P_i , and o_i denote the task situation, the associated plan, and the execution outcome of completed mission i , respectively. Validation feedback, runtime perturbations, and reusable guidance are retained as associated information. The system retrieves the K most semantically similar entries to the current query according to

$$\begin{aligned} \mathcal{R}_K(q, M) &= \underset{m_i \in M}{\text{TopK}} \text{ sim}(q, m_i), \\ \text{sim}(q, m_i) &= \frac{\phi(q)^\top \phi(m_i)}{\|\phi(q)\|_2 \|\phi(m_i)\|_2}. \end{aligned} \quad (3)$$

where $\mathcal{R}_K(q, M)$ is the set of the K entries retrieved from M for query q , $\phi(\cdot)$ maps a query or memory entry to its semantic embedding. The retrieved set provides Mission Planning or Runtime Replanning with concrete tactical precedents and reusable guidance about recurring planning, resource-management, and repair patterns.

III. EVALUATION SUITE DESIGN

This section presents a controlled, real-UAV-informed evaluation suite in AirSim for testing whether CLoMA-UWR remains effective as operational pressure increases. It instantiates the task model in Section II through real-to-sim calibration, difficulty stratification, and mission-level metrics, which ground the simulated setting, vary operational pressure, and separate task completion, execution quality, and mission continuity. As summarized in Table I, existing settings cover only parts of the transition from aerial observation to task-zone construction, resource-constrained mission planning, and runtime replanning; the comparison concerns evaluation coverage rather than algorithmic performance. The following subsections detail the three design elements.

A. Real-to-Sim Calibration

The first design property is realized through a co-design between the real UAV platform and the AirSim simulation

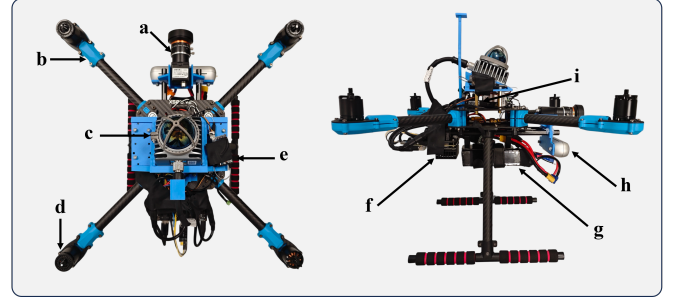


Fig. 2. UAV platform and its main components: (a) LiDAR, (b) frame, (c) monocular camera, (d) motor, (e) receiver, (f) onboard computer, (g) power supply, (h) stereo camera, and (i) flight controller.

environment. As stated above, real UAV parameters constrain every evaluation episode. Accordingly, the real platform defines the engineering boundary of the rescue mission, while AirSim configures controllable and repeatable evaluation episodes within that boundary. Rather than treating the two as separate experimental settings, we use the real platform to define the feasible set from which AirSim configures each simulated UAV and its environment. For each UAV, the configured mission must satisfy

$$\begin{aligned} \mathcal{E}_{\text{UAV}} &= \{(h, v, m, e_{\text{avail}}, r) \mid \\ &h \in \mathcal{H}_{\text{safe}}, \quad v \in \mathcal{V}_{\text{safe}}, \\ &m \leq m_{\text{max}}, \quad e_{\text{req}} + e_{\text{res}} \leq e_{\text{avail}}, \\ &r \leq r_{\text{link}}\}. \end{aligned} \quad (4)$$

where $\mathcal{E}_{\text{UAV}}$ is the feasible UAV configuration set; h , v , m , e_{avail} , and r denote flight altitude, velocity, payload mass, available energy, and communication-link distance, respectively. $\mathcal{H}_{\text{safe}}$ and $\mathcal{V}_{\text{safe}}$ are the safe altitude and velocity sets specified by the operating policy; m_{max} is the maximum payload mass; e_{req} and e_{res} are the energy required for the mission and the reserve energy for a safe return; and r_{link} is the maximum communication-link distance. These bounds combine hardware specifications, flight-safety policy, and deployment configuration. Any configuration that violates Eq. 4 is excluded before episode generation. The real UAV platform is shown in Figure 2, and its hardware specifications are listed in Table II.

The virtual observation process is further fixed by the real camera geometry. Under a nadir-view, locally planar-ground approximation, for altitude h , horizontal field of view θ_{FOV} , and horizontal resolution N_x , the image footprint and ground sampling distance are

$$W(h) = 2h \tan\left(\frac{\theta_{\text{FOV}}}{2}\right), \quad \text{GSD}(h) = \frac{W(h)}{N_x}. \quad (5)$$

Thus, the AirSim camera pose and the scale, density, and placement of fire points are jointly constrained by what the real camera could observe, rather than selected independently. Energy and payload limits likewise constrain route length, task duration, and suppressible workload. Table III records these calibration rules and the resulting simulation settings.

These calibrated bounds provide the common operational basis on which the difficulty levels are constructed.

TABLE II
TECHNICAL SPECIFICATIONS OF THE REAL UAV PLATFORM.

Subsystem	Model	Key specifications
UAV frame	Holybro X500 V2	Carbon-fiber quadrotor frame, 500-mm wheelbase, approximately 610-g frame weight, and 1.5-kg published payload capacity excluding the battery at 70% throttle.
Flight controller	Holybro Pixhawk 6X	STM32H753 processor at 480 MHz, triple-redundant IMUs, dual barometers, 16 PWM outputs, two CAN buses, and 100-Mbps Ethernet.
Companion computer	NVIDIA Jetson Orin NX 8 GB	6-core Arm Cortex-A78AE CPU, NVIDIA Ampere GPU, 8-GB LPDDR5 memory, up to 117 TOPS, and configurable 10–40-W power modes.
LiDAR	Livox Mid-360	360° horizontal and -7° – 52° vertical field of view, 200,000 points/s, 10-Hz frame rate, 40-m range at 10% reflectivity, 0.1-m blind zone, 265-g weight, and 6.5-W average power.
RGB-D camera	Intel RealSense D455	Global-shutter stereo depth camera, $87^\circ \times 58^\circ$ depth field of view, depth resolution up to 1280×720 at 90 fps, approximately 0.6–6-m effective range, integrated IMU, USB 3.1 interface, and 116-g weight.
Propulsion system	4× Holybro 2216 KV920	Four 920-KV brushless motors, four 20-A ESCs, and 10×4.5 -in propellers, with a maximum manufacturer bench thrust of approximately 1.33 kgf per motor at 16 V.
Battery	Dualsky XP3300-6HED	6S lithium-polymer battery, 3300-mAh capacity, 22.2-V nominal voltage, 25.2-V fully charged voltage, and approximately 73.3-Wh nominal energy.

TABLE III
REAL-TO-SIM CALIBRATION OF AIRSIM UAV AND ENVIRONMENT SETTINGS.

Calibration target	Real-platform basis	AirSim configuration
Imaging geometry	Operating altitude and the RGB-D camera field of view and resolution	Observation height h , camera pose, field of view, and resolution; the resulting footprint $W(h)$ and GSD determine the observable ground area and image scale.
Energy model	Battery capacity, propulsion characteristics, and return-to-base reserve policy	Available energy e_{avail} , reserve e_{res} , and energy consumption for flight and task execution.
Payload model	Airframe and propulsion payload limit	Maximum payload m_{max} and the onboard suppressant inventory for each simulated UAV.
Communication model	Deployed communication configuration and operating range	Communication-link distance threshold r_{link} and the distance-dependent connectivity condition.

TABLE IV
FOUR DIFFICULTY LEVELS AND THEIR CONTROLLED EPISODE CONDITIONS.

Difficulty levels	Fire-point configuration	Fleet/resource conditions	Runtime perturbations
Simple	Sparse static fire points; few task zones.	Ample energy and payload.	None.
Normal	Multiple fire points and task zones.	Multi-UAV allocation; normal resource margin.	None.
Hard	Multi-region fire scene.	Energy or payload constrained.	Few, temporally separated perturbations.
Expert	Dense or spreading fire-point layout.	Competing resource demands.	Frequent perturbations; global scheduling pressure.

B. Difficulty Stratification

Each of the four difficulty levels in Table IV contains 20 fixed evaluation episodes, yielding 80 episodes in total. The simple and normal levels test fire-situation understanding and resource-constrained mission planning without runtime perturbations, whereas the hard and expert levels introduce progressively stronger resource constraints and scheduling interactions together with runtime perturbations. All fleet, sensing, energy, payload, and communication settings remain within the real-to-sim calibrated bounds in Section III-A.

The fire-point configurations of all episodes are pre-specified rather than randomly generated. These configurations control the scene structure and the spatial arrangement and number of fire points per task zone across difficulty levels. Figure 3 provides representative scenes from the four difficulty levels, and Figure 4 summarizes the distribution of fire-point counts per task zone within each difficulty level. Together, they illustrate

the scene layouts and task-zone-level fire-point distributions used to instantiate the four difficulty levels.

Runtime perturbations modify the operational state or mission constraints after execution begins. Figure 5 summarizes the five event families and the fixed draws used in hard and expert episodes; events are sampled once during suite construction and placed at predefined episode points, with hard episodes using a few temporally separated perturbations and expert episodes using more frequent, potentially overlapping perturbations.

C. Evaluation Metrics

We evaluate end-to-end behavior using six complementary mission-level metrics: completion, response, mission time, event stability, resource pressure, and coordination. They measure objective attainment, response timeliness, execution efficiency, continuity under perturbations, resource use, and workload balance, respectively. Event stability is evaluated only

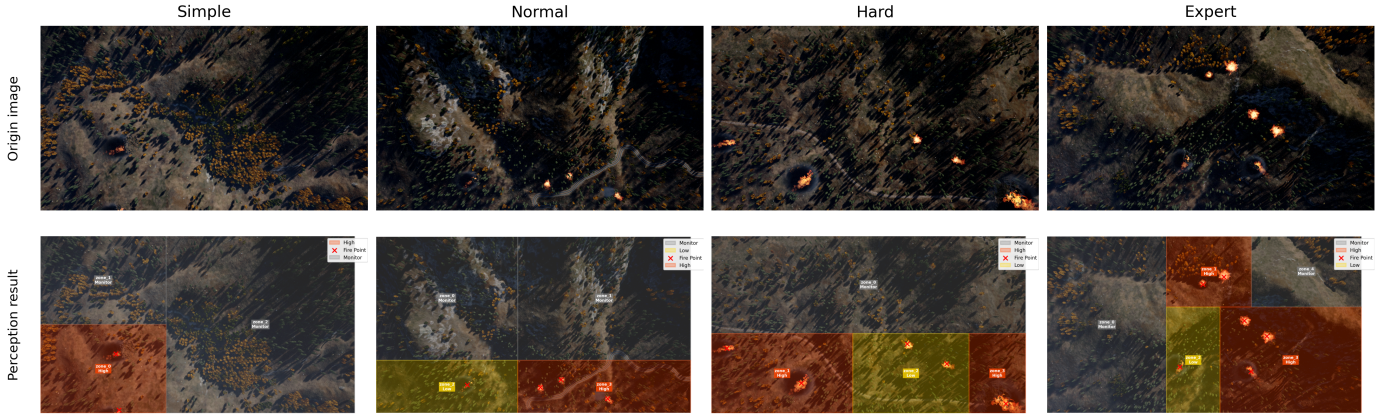


Fig. 3. Representative fire scenes from the four difficulty levels. In each example, *Origin image* denotes the original aerial image, and *Perception result* denotes the output generated by the Vision Analysis module, which uses Gemini 3.1 Pro as its foundation model.

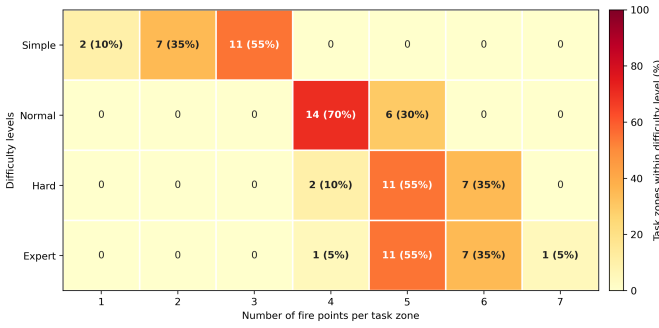


Fig. 4. Distribution of fire-point counts per task zone across difficulty levels.

in the hard and expert settings, where runtime perturbations are introduced.

All scores are normalized to $[0, 100]$, with higher values indicating better performance. Let Z be the set of task zones, $c_z \in [0, 1]$ the completion ratio of zone z , and ω_z its priority weight. Let $\bar{t}_{\text{resp}}$ and τ denote the priority-weighted mean time to first effective intervention and the total mission duration. The six scores are defined jointly as

$$\begin{aligned}
 s_{\text{comp}} &= 100 \frac{\sum_{z \in Z} \omega_z c_z}{\sum_{z \in Z} \omega_z}, \\
 s_{\text{resp}} &= 100 \left[1 - \frac{\bar{t}_{\text{resp}} - l_{\text{resp}}}{u_{\text{resp}} - l_{\text{resp}}} \right]_0^1, \\
 s_{\text{time}} &= 100 \left[1 - \frac{\tau - l_{\text{time}}}{u_{\text{time}} - l_{\text{time}}} \right]_0^1, \\
 s_{\text{stab}} &= \frac{100}{N_e} \sum_{e=1}^{N_e} \mathbb{I}(\text{Recovered}(e)), \\
 s_{\text{res}} &= 100 \left[1 - \frac{\bar{p} + \max_j p_j}{2} \right], \\
 s_{\text{coord}} &= 100 \left[1 - \frac{G(\mathbf{n}) + G(\mathbf{d}) + G(\boldsymbol{\tau})}{3} \right].
 \end{aligned} \tag{6}$$

Here, s_{comp} , s_{resp} , s_{time} , s_{stab} , s_{res} , and s_{coord} denote the completion, response, mission time, event stability, resource pressure, and coordination scores, respectively. $[x]_0^1 =$

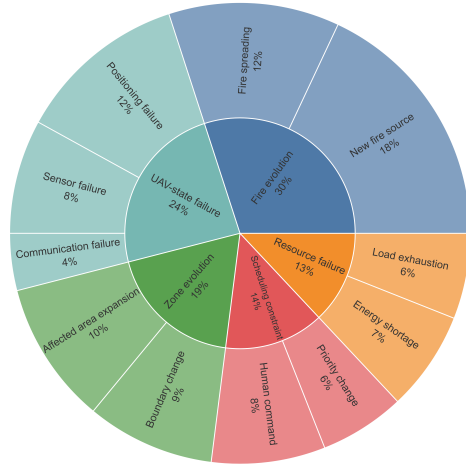
$\min\{1, \max\{0, x\}\}$ clips a value x to $[0, 1]$; l_{resp} , u_{resp} , l_{time} , and u_{time} are fixed time-reference thresholds. N_e is the number of runtime perturbations, e indexes these perturbations, and $\text{Recovered}(e)$ indicates recovery from perturbation e by a feasible revision within the allowed window. $\mathbb{I}(\cdot)$ is the indicator function, which equals 1 when its condition is true and 0 otherwise. $p_j \in [0, 1]$ and $\bar{p}$ denote the resource pressure of UAV j and the fleet-average resource pressure, respectively, where j indexes the UAVs in the fleet. $G(\cdot) \in [0, 1]$ is the Gini coefficient of the assigned task counts $\mathbf{n}$, flight distances $\mathbf{d}$, or execution durations $\boldsymbol{\tau}$.

Because no perturbations are introduced in the simple and normal settings, their applicable metric set $\mathcal{K}_d$ contains completion, response, mission time, resource pressure, and coordination; for the hard and expert settings, it additionally contains event stability. For each episode i at difficulty level d , the score combines the weighted aggregation of its applicable metrics with a bounded difficulty reward. The reported score for level d is the mean across its N_d episodes:

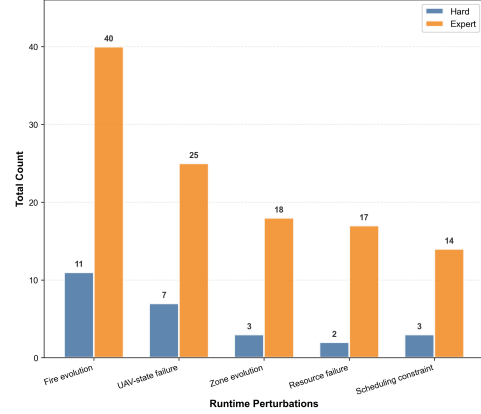
$$\begin{aligned}
 S_d &= \frac{1}{N_d} \sum_{i=1}^{N_d} \left[\sum_{k \in \mathcal{K}_d} w_{k,d} s_{i,k} \frac{1 + \lambda \rho_i}{1 + \lambda} \right], \\
 w_{k,d} &\geq 0, \quad \sum_{k \in \mathcal{K}_d} w_{k,d} = 1.
 \end{aligned} \tag{7}$$

Here, $s_{i,k}$ is the score of episode i for metric k , and $w_{k,d}$ is its normalized weight at difficulty level d . The coefficient $\rho_i \in [0, 1]$ represents episode difficulty, while $\lambda \geq 0$ controls the difficulty reward; division by $1 + \lambda$ keeps S_d within $[0, 100]$. The component scores support diagnostic analysis, whereas S_d provides the overall comparison score for level d .

Together, the staged episode difficulty and decomposed metrics provide a common basis for testing the three questions examined next: whether CLoMA-UWR sustains high-level closed-loop decision making, which mechanisms matter as operational pressure increases, and how much capability for resource-constrained mission planning can be retained by a compact planner.



(a) Runtime perturbation library.



(b) Distribution of perturbations in hard and expert levels.

Fig. 5. Runtime perturbations used for closed-loop evaluation.

IV. EXPERIMENTS

This section evaluates CLoMA-UWR with the evaluation suite in Section III through three settings. The first examines whether CLoMA-UWR outperforms an Optimization-Based Planner as operational pressure increases and how sensitive it is to model choice. The second isolates how Validation, Memory, and Runtime Replanning address their respective decision risks, while the third tests whether validated demonstrations can specialize a compact planner with lower planning overhead. The following subsections specify the experimental setup and then report the end-to-end, ablation, and fine-tuned planner evaluations.

A. Experimental Setup

1) *End-to-End Setting*: We fix Gemini 3.1 Pro for Vision Analysis and use ChatGPT 5.5, Gemini 3.1 Pro, Claude Opus 4.7, or DeepSeek V4 Pro in Mission Planning, Validation, and Runtime Replanning. Fixing Vision Analysis keeps the structured fire situation consistent, so differences among the four downstream foundation-model configurations reflect the downstream decision process.

The Optimization-Based Planner is implemented with the CP-SAT solver in Google OR-Tools. It shares the Vision Analysis output, UAV states, mission constraints, and action space with CLoMA-UWR, and re-optimizes the remaining task chains from the updated operational state after each runtime perturbation. Its downstream decision process does not use an LLM, Validation, or Memory. The baseline and four foundation-model configurations use the same 80 episodes—20 at each difficulty level—and the same execution and scoring procedure.

2) *Ablation Setting*: Using the DeepSeek V4 Pro configuration, we compare CLoMA-UWR with w/o Validation, w/o Memory, and w/o Runtime Replanning. These variants respectively remove Validation, disable retrieval from Memory, and retain the initial plan after runtime perturbations. All other models, prompts, and execution settings remain unchanged, and all four configurations use the same 80 episodes, with 20 at each difficulty level. Because simple and normal episodes contain no

runtime perturbations, Runtime Replanning remains inactive; the w/o Runtime Replanning variant therefore introduces no other execution difference at these levels.

3) *Fine-Tuning Data and Training*: We construct 2,000 mission-planning demonstrations using CLoMA-UWR with Gemini 3.1 Pro as the teacher planner from mission situations disjoint from the 80 evaluation episodes. Each demonstration maps (R, U, C) to P , pairing the task zones, UAV states, and mission constraints with the final multi-UAV plan accepted by Validation. These validated demonstrations specialize Qwen3-8B for the planning mapping in Section II. We use a 90%/10% training/validation split and 4-bit quantized low-rank adaptation with a maximum sequence length of 6,144 tokens, three epochs, an effective batch size of 16, and a learning rate of 5×10^{-5} . Fine-tuning uses one NVIDIA GeForce RTX 4090, 16 Intel Xeon Gold 6430 vCPUs, and 120 GB of system memory.

4) *Planner Evaluation Protocol*: We evaluate the original and fine-tuned Qwen3-8B by replacing only the planner used by Mission Planning. Vision Analysis and Validation retain Gemini 3.1 Pro, while all non-LLM components remain unchanged. Both configurations use the same 40 simple and normal episodes—20 at each level—from the first setting. Because these episodes contain no runtime perturbations, the experiment neither invokes Runtime Replanning nor assesses event stability, thereby isolating planner fine-tuning.

B. End-to-End Evaluation

Table V reports the end-to-end results, while Figures 6 and 7 summarize the overall scores and mission-level profiles.

In the simple and normal settings, where no runtime perturbations occur, all evaluated methods achieve complete task coverage. The Optimization-Based Planner obtains overall scores of 92.59 and 88.72, respectively, which lie within the corresponding ranges of the four foundation-model configurations (90.54–93.23 and 86.28–89.18), confirming that it remains competitive under lower operational pressure.

Once runtime perturbations are introduced, the difference between the two decision approaches becomes more pronounced:

TABLE V

END-TO-END RESULTS FOR THE OPTIMIZATION-BASED PLANNER AND CLOMA-UWR ACROSS EPISODE DIFFICULTY LEVELS. OPTIMIZATION-BASED DENOTES THE OPTIMIZATION-BASED PLANNER, WHILE EACH MODEL NAME DENOTES THE CORRESPONDING DOWNSTREAM FOUNDATION-MODEL CONFIGURATION OF CLOMA-UWR. S IS THE OVERALL SCORE, AND THE REMAINING COLUMNS REPORT NORMALIZED MISSION-LEVEL SCORES IN $[0, 100]$ (HIGHER IS BETTER). EVENT STABILITY IS NOT APPLICABLE TO THE SIMPLE AND NORMAL SETTINGS AND IS DENOTED BY “—”. THE HIGHEST OBSERVED VALUE FOR EACH METRIC AT EVERY DIFFICULTY LEVEL IS SHOWN IN BOLD WHEN IT IS UNIQUE; THESE HIGHLIGHTS DESCRIBE THE EVALUATED CONFIGURATIONS RATHER THAN A GENERAL RANKING OF FOUNDATION MODELS.

Difficulty	Configuration	S	Mission-level score					
			Completion	Response	Time	Event stability	Resource pressure	Coordination
Simple	Optimization-Based	92.59	100.00	92.34	88.94	—	85.64	82.31
	ChatGPT 5.5	90.63	100.00	89.18	78.99	—	81.20	79.54
	Gemini 3.1 Pro	93.23	100.00	88.78	91.06	—	84.45	80.71
	Claude Opus 4.7	92.97	100.00	91.44	86.62	—	82.18	83.96
	DeepSeek V4 Pro	90.54	100.00	87.66	78.90	—	83.05	80.04
Normal	Optimization-Based	88.72	100.00	89.07	72.71	—	87.05	77.74
	ChatGPT 5.5	86.28	100.00	93.82	66.27	—	77.64	73.19
	Gemini 3.1 Pro	88.23	100.00	94.10	71.50	—	86.68	71.72
	Claude Opus 4.7	89.18	100.00	96.09	68.24	—	84.54	78.12
	DeepSeek V4 Pro	87.63	100.00	91.36	71.74	—	83.95	75.13
Hard	Optimization-Based	49.77	96.40	78.32	26.94	20.75	54.86	52.14
	ChatGPT 5.5	65.09	100.00	94.97	41.22	42.25	63.19	67.24
	Gemini 3.1 Pro	62.81	100.00	92.58	39.60	33.75	59.10	60.65
	Claude Opus 4.7	64.74	99.86	94.22	35.28	40.50	62.15	65.88
	DeepSeek V4 Pro	59.03	98.57	83.90	30.76	34.75	61.36	59.07
Expert	Optimization-Based	21.53	78.27	42.13	1.93	0.00	42.07	39.87
	ChatGPT 5.5	40.15	93.24	67.01	13.23	2.67	60.15	61.25
	Gemini 3.1 Pro	38.28	90.09	61.30	12.02	0.00	58.72	55.82
	Claude Opus 4.7	39.81	92.43	65.55	9.13	0.00	58.99	60.11
	DeepSeek V4 Pro	37.55	90.95	61.09	3.54	0.00	53.43	54.43

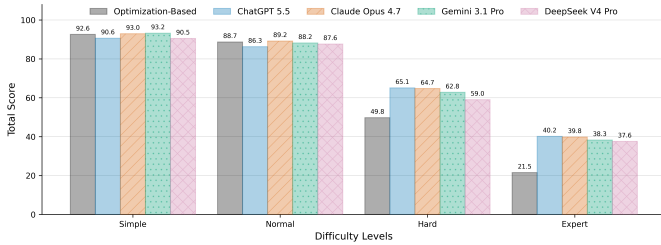


Fig. 6. Overall scores of the Optimization-Based Planner and four foundation-model configurations across episode difficulty levels.

the Optimization-Based Planner scores 49.77 and 21.53 in the hard and expert settings, respectively 9.26 and 16.02 points below the lowest-scoring foundation-model configuration. Its expert completion score is 78.27, compared with 90.09–93.24 for the four configurations. Because its event stability score of 0 overlaps their 0–2.67 range, the advantage of CLOMA-UWR also reflects task completion and the other execution-quality metrics.

Among the four foundation-model configurations, Gemini 3.1 Pro obtains the highest overall score in the simple setting, Claude Opus 4.7 in the normal setting, and ChatGPT 5.5 in the hard and expert settings. Because this ordering changes with difficulty, the results do not yield a single model ranking. All four configurations maintain high basic task completion in the hard and expert episodes, but their mission time scores fall to 3.54–13.23 and their event stability scores to 0–2.67 under

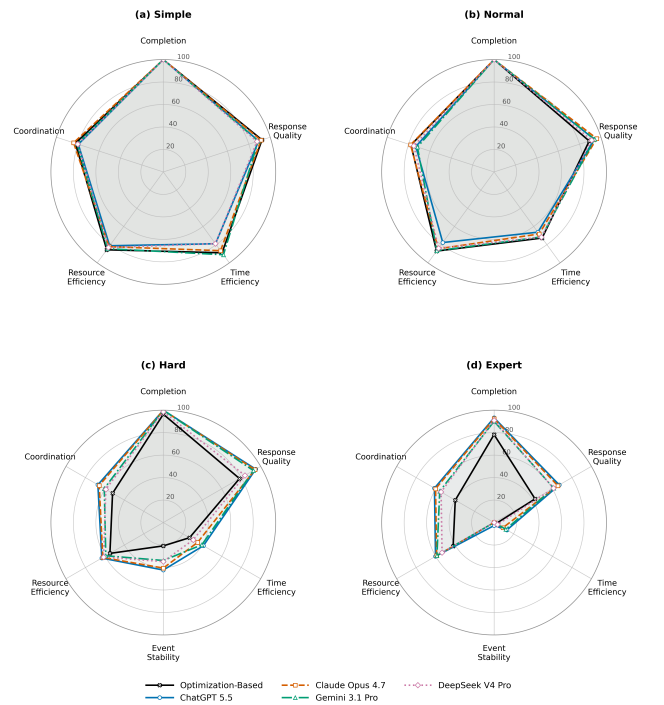


Fig. 7. Mission-level performance profiles of the Optimization-Based Planner and four foundation-model configurations at each difficulty level. Larger radial values indicate better performance.

TABLE VI
ABLATION RESULTS WITH DEEPSEEK V4 PRO AS THE FOUNDATION MODEL.
VALUES REPORT THE MEAN OVERALL SCORE S OVER 20 EPISODES AT
EACH DIFFICULTY LEVEL. THE BEST RESULT IN EACH COLUMN IS SHOWN
IN BOLD.

Configuration	Overall score S			
	Simple	Normal	Hard	Expert
Full	90.54	87.63	59.03	37.55
w/o Validation	88.38	85.30	55.22	35.50
w/o Memory	88.57	86.94	49.35	22.10
w/o Runtime Replanning	90.33	88.14	35.86	12.98

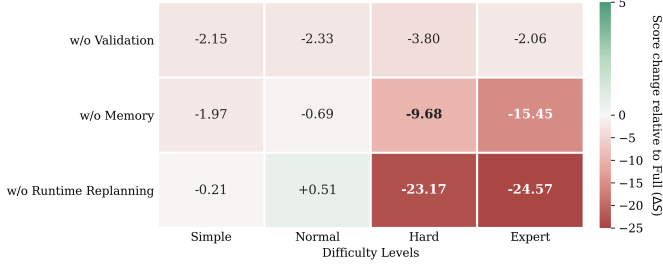


Fig. 8. Change in overall score relative to CLOMA-UWR for each ablated variant and difficulty level. Negative values indicate performance degradation, while positive values indicate improvement.

expert conditions. Model choice thus affects the downstream decision process, but changing the foundation model alone does not prevent the common degradation as operational pressure increases.

Taken together, the Optimization-Based Planner remains competitive in the simple and normal episodes, where no runtime perturbations occur. Under the higher operational pressure of the hard and expert episodes, CLOMA-UWR maintains higher basic task completion and execution quality across all four downstream foundation-model configurations. The common decline in mission time and event stability under expert conditions nevertheless leaves clear headroom for more robust replanning and time-aware resource allocation.

C. Ablation Studies

Table VI reports the ablation results with DeepSeek V4 Pro fixed as the foundation model. Removing Validation causes a modest but consistent reduction across all four difficulty levels, with drops of 2.16, 2.33, 3.81, and 2.05 points from simple to expert. The limited change in average mission score does not imply that the module remains inactive: the validation loop is triggered 5, 5, 4, and 6 times in the simple, normal, hard, and expert settings, respectively. These interventions suggest that Validation primarily serves as an occasional reliability safeguard against invalid or tactically implausible plans rather than as a continuous source of score improvement.

Figure 8 visualizes the score change of each ablated variant relative to CLOMA-UWR.

The contribution of Memory becomes more pronounced as operational pressure increases. Removing Memory reduces the score by only 1.97 points in the simple setting and 0.69 points

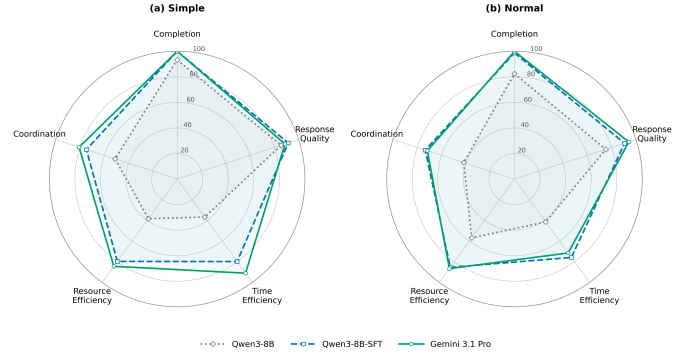


Fig. 9. Mission-level profiles of the three planners in the simple and normal settings. Larger radial values indicate better performance. Event stability is not interpreted because neither setting contains runtime perturbations.

in the normal setting, but the reduction grows to 9.68 points in hard episodes and 15.45 points in expert episodes. This pattern indicates that retrieved cases and distilled lessons are most useful when resource constraints, event interactions, and scheduling pressure make planning less routine.

Disabling Runtime Replanning has little effect in the simple and normal settings, where no perturbations are triggered. In contrast, its score falls from 59.03 to 35.86 in hard episodes and from 37.55 to 12.98 in expert episodes. The sharp degradation confirms that an initially valid plan is insufficient once the execution state changes, and that Runtime Replanning is the key mechanism for maintaining mission performance under perturbations.

The ablations therefore do not indicate three mechanisms that contribute uniformly in every episode. Validation acts as a low-frequency safeguard against invalid candidate plans, Memory contributes most when operational pressure makes planning less routine, and Runtime Replanning becomes critical when runtime perturbations invalidate an accepted plan. Their condition-dependent effects explain why all three are retained in CLOMA-UWR despite their different average score contributions.

D. Fine-Tuned Planner Evaluation

Table VII compares the Gemini 3.1 Pro teacher planner with the original and fine-tuned Qwen3-8B planners. Fine-tuning substantially improves the compact planner on both difficulty levels, yielding an average absolute gain of 23.65 points and reaching 98.24% of the Gemini 3.1 Pro teacher planner's average score across the simple and normal settings.

Figure 9 shows that the improvement extends across the mission-level performance profile. The original Qwen3-8B exhibits its largest deficits in mission time, resource pressure, and coordination, whereas fine-tuning substantially expands its profile toward that of Gemini 3.1 Pro. The fine-tuned planner restores near-complete task coverage and slightly exceeds Gemini 3.1 Pro in simple-level response and normal-level mission time and coordination. This indicates that the validated demonstrations transfer not only output-format compliance but also task-allocation and resource-management behavior to the compact planner.

TABLE VII

COMPARISON OF THE GEMINI 3.1 PRO TEACHER PLANNER, THE ORIGINAL QWEN3-8B PLANNER, AND THE FINE-TUNED QWEN3-8B PLANNER. MISSION-LEVEL SCORES ARE NORMALIZED TO $[0, 100]$ AND HIGHER IS BETTER. FOR PLANNING TIME AND TOKEN USAGE, LOWER IS BETTER. THE BEST VALUE AT EACH DIFFICULTY LEVEL IS SHOWN IN BOLD WHEN IT IS UNIQUE.

Difficulty	Planner	S	Mission-level score					Planning overhead	
			Completion	Response	Time	Resource pressure	Coordination	Time (s)	Tokens
Simple	Gemini 3.1 Pro	93.23	100.00	88.78	91.06	84.45	80.71	33.47	11,730
	Qwen3-8B	63.77	93.34	85.48	36.77	38.33	51.07	18.37	3,804
	Qwen3-8B-SFT	90.87	100.00	91.44	79.83	79.66	74.73	12.95	3,485
Normal	Gemini 3.1 Pro	88.23	100.00	94.10	71.50	86.68	71.72	41.53	14,975
	Qwen3-8B	67.19	82.50	75.17	41.50	56.91	41.78	17.52	3,844
	Qwen3-8B-SFT	87.39	98.75	90.57	75.83	85.45	73.43	15.07	3,701

The fine-tuned planner also reduces planning overhead under the evaluated deployment setting. Its mean planning time across the two levels is 14.01 s, compared with 37.50 s for Gemini 3.1 Pro, while its mean planning-token usage falls from 13,353 to 3,593. Figure 10 summarizes this performance–efficiency trade-off. This evidence is limited to initial planning on the evaluated simple and normal episodes; it does not establish how the compact planner would perform with Runtime Replanning under hard or expert conditions.

V. CONCLUSION

Within the task and evaluation setting studied here, this work examines the contribution of LLMs to resource-constrained multi-UAV wildfire rescue through CLoMA-UWR, which connects fire-situation understanding and resource-constrained mission planning with Validation, Memory, and Runtime Replanning. This integration enables LLMs to support a continuous rescue workflow that extends from interpreting the fire situation and coordinating a resource-limited fleet to adapting an active mission after operational state changes.

Across four downstream foundation-model configurations, CLoMA-UWR sustains high basic task completion as operational pressure increases, although execution quality and mission continuity deteriorate in harder episodes. The Optimization-Based Planner provides a non-LLM reference under the same

evaluation protocol. The ablations further reveal condition-dependent roles: Validation provides an occasional reliability safeguard, Memory becomes increasingly valuable as resource and scheduling interactions intensify, and Runtime Replanning is critical when perturbations invalidate an active plan. Meanwhile, Qwen3-8B fine-tuned with validated planning demonstrations retains 98.24% of the Gemini 3.1 Pro teacher planner’s average score on simple and normal episodes, while reducing mean planning time by approximately 63% and planning-token usage by 73%. This supports lower-cost local deployment of Mission Planning within the evaluated setting.

These answers remain limited to a calibrated simulation, a fixed episode set, and one real-platform configuration. Moreover, the compact planner has not yet been evaluated under hard and expert runtime conditions. The evaluation also does not include a direct numerical comparison with an external end-to-end rescue method because existing systems assume different inputs, action spaces, resource constraints, and evaluation protocols. The reported evidence therefore supports CLoMA-UWR within the task and evaluation setting defined here. Future work will extend the evaluation to real-flight and mixed real–sim trials, more diverse platforms and fire dynamics, and larger fleets, while developing more selective memory retrieval and more robust, efficient replanning for missions under high operational pressure.

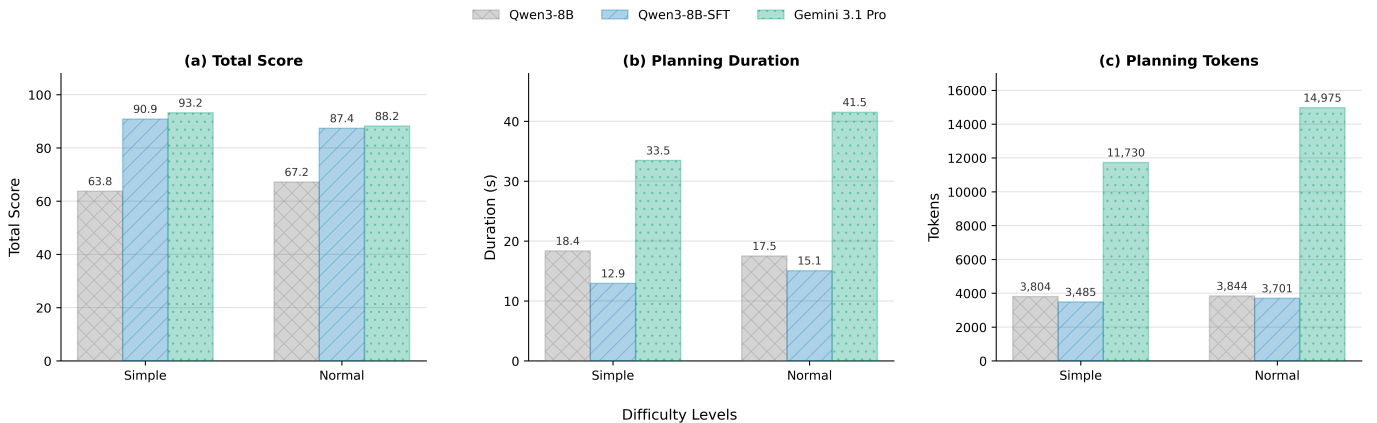


Fig. 10. Score–efficiency comparison of the three planners: (a) total score, (b) planning duration, and (c) planning-token usage. Lower values are better in (b) and (c).

REFERENCES

- [1] D. M. J. S. Bowman *et al.*, “Human exposure and health impacts of wildfire smoke,” *Environmental Research Letters*, vol. 15, 2020.
- [2] J. Allan *et al.*, “Remote sensing of wildfires: A review of UAV and satellite-based approaches,” *Remote Sensing*, 2022.
- [3] D. Floreano and R. J. Wood, “Science, technology and the future of small autonomous drones,” *Nature*, vol. 521, pp. 460–466, 2015.
- [4] M. Erdelj, M. Król, and E. Natalizio, “Help from the sky: UAV applications in disaster management,” *IEEE Transactions on Mobile Computing*, vol. 16, no. 9, pp. 2675–2688, 2017.
- [5] F. Yuan *et al.*, “A deep learning framework for fire detection using UAV-based thermal imaging and RGB data,” *IEEE Transactions on Geoscience and Remote Sensing*, 2021.
- [6] Y. Zhang *et al.*, “Transformer-based semantic segmentation for UAV remote sensing images,” *IEEE Transactions on Geoscience and Remote Sensing*, 2022.
- [7] Z. Yang *et al.*, “Learning-based UAV 3D reconstruction for autonomous inspection and mapping,” *IEEE Robotics and Automation Letters*, 2022.
- [8] M. Popovic *et al.*, “Autonomous UAV exploration and mapping for disaster response,” *IEEE Robotics and Automation Letters*, 2021.
- [9] A. Bircher, M. Kamel, K. Alexis, H. Oleynikova, and R. Siegwart, “Receding horizon “Next-Best-View” planner for 3D exploration,” in *Proceedings of the IEEE International Conference on Robotics and Automation*. IEEE, 2016, pp. 1462–1468.
- [10] S. Xiao *et al.*, “Deep reinforcement learning for UAV autonomous navigation in unknown environments,” *IEEE Transactions on Intelligent Transportation Systems*, 2021.
- [11] C. Yu *et al.*, “Coverage path planning for UAV-based search and rescue missions,” *IEEE Transactions on Automation Science and Engineering*, 2022.
- [12] Y. Yang *et al.*, “Multi-agent reinforcement learning for cooperative UAV navigation and task allocation,” *IEEE Transactions on Neural Networks and Learning Systems*, 2023.
- [13] S. Waharte and N. Trigoni, “Supporting search and rescue operations with UAVs,” *Survey Literature on UAV-based Disaster Response*, 2021.
- [14] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn *et al.*, “Do as I can, not as I say: Grounding language in robotic affordances,” in *Conference on Robot Learning*, 2022, pp. 287–318.
- [15] J. Liang, W. Huang *et al.*, “Code as policies: Language model programs for embodied control,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2023.
- [16] S. Vemprala, R. Bonatti, A. Buckner, and A. Kapoor, “ChatGPT for robotics: Design principles and model abilities,” *Microsoft Research Technical Report*, 2023.
- [17] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, C. Finn, K. Hausman *et al.*, “RT-1: Robotics transformer for real-world control at scale,” in *Robotics: Science and Systems*, 2023.
- [18] D. Driess, F. Xia, M. S. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter *et al.*, “PaLM-E: An embodied multimodal language model,” in *Proceedings of the International Conference on Machine Learning*, 2023.
- [19] A. Zeng, A. Wong *et al.*, “Socratic models: Composing zero-shot multimodal reasoning with language models,” in *Proceedings of the International Conference on Learning Representations*, 2023.
- [20] W. Huang, F. Xia *et al.*, “Inner monologue: Embodied reasoning through planning with language models,” in *Conference on Robot Learning*, 2023.
- [21] D. Shah, M. Equi *et al.*, “LM-Nav: Robotic navigation with large pre-trained models,” in *Conference on Robot Learning*, 2023.
- [22] G. Wang, Y. Xie *et al.*, “Voyager: An open-ended embodied agent with large language models,” in *Advances in Neural Information Processing Systems*, 2023.
- [23] G.-S. Xia, X. Bai, J. Ding, Z. Zhu, S. Belongie, J. Luo, M. Datcu, M. Pelillo, and L. Zhang, “DOTA: A large-scale dataset for object detection in aerial images,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 3974–3983.
- [24] P. Zhu, L. Wen, D. Du, X. Bian, H. Fan, Q. Hu, and H. Ling, “Detection and tracking meet drones challenge,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7380–7399, 2022.
- [25] M. N. Mowla, D. Asadi, K. N. Tekeoglu, S. Masum, and K. Rabie, “UAVs-FFDB: A high-resolution dataset for advancing forest fire detection and monitoring using unmanned aerial vehicles (UAVs),” *Data in Brief*, vol. 55, p. 110706, 2024.
- [26] A. Shamsoshara, F. Afghah, A. Razi, L. Zheng, P. Z. Fulé, and E. Blasch, “Aerial imagery pile burn detection using deep learning: The FLAME dataset,” *Computer Networks*, vol. 193, p. 108001, 2021.
- [27] X. Han, N. Pu, Z. Feng, Y. Bei, Q. Zhang, L. Cheng, and L. Xue, “Benchmarking multi-scene fire and smoke detection,” in *Pattern Recognition and Computer Vision: 7th Chinese Conference, PRCV 2024, Urumqi, China, October 18–20, 2024, Proceedings, Part XI*, ser. Lecture Notes in Computer Science, vol. 15041. Springer Nature Singapore, 2024, pp. 203–218.
- [28] M. Habibpour, N. Alipour Talemi, J. Spodnik, C. J. Khoury, and F. Afghah, “WildFireVQA: A large-scale radiometric thermal VQA benchmark for aerial wildfire monitoring,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2026.
- [29] A. Kleiner, A. Farinelli, S. Ramchurn, B. Shi, F. Maffioletti, and R. Reffato, “RMASBench: Benchmarking dynamic multi-agent coordination in urban search and rescue,” in *Proceedings of the 12th International Conference on Autonomous Agents and Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, 2013, pp. 1195–1196, extended abstract.
- [30] F. Alam, H. Sajjad, M. Imran, and F. Ofli, “CrisisBench: Benchmarking crisis-related social media datasets for humanitarian information processing,” in *Proceedings of the International AAAI Conference on Web and Social Media*, vol. 15, 2021, pp. 923–932.